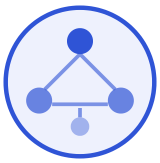
GUÍA PRÁCTICA · NEIRO AGENCY

MiroFish: simular cómo reaccionaría la gente, y qué tan útil es para probar webs



MiroFish es un proyecto de código abierto que crea un “mundo paralelo” de agentes de IA que publican, comentan y reaccionan sobre un tema que vos le das. Esta guía explica cómo funciona, qué necesita para correr, qué sale de una simulación y, sobre todo, hasta dónde sirve para probar landings y desarrollos y dónde no, según lo que dice su propio código.

Información al 23 de octubre de 2026. Este proyecto cambia rápido, así que lo que se describe aquí puede variar con el tiempo.

Marcos Vargas IA · Sistemas · Automatización

LO QUE APRENDERÁS

01	Qué es MiroFish y cómo trabaja una simulación
02	Qué necesita y cómo instalarlo
03	Qué entra, qué sale y qué simula exactamente
04	Si sirve para probar landings y desarrollos
05	Lo bueno del proyecto
06	Riesgos y límites que no se ven en la demo
07	Cómo lo uso yo

SECCIÓN 01
QUÉ ES

Un simulador de opinión en redes sociales con agentes de IA.

El repositorio es **666ghj/MiroFish** (licencia AGPL-3.0; versión 0.1.0, y su interfaz se presenta como “v0.1-Preview”). Se describe como un “motor de inteligencia de enjambre” para predecir cualquier cosa y, en el texto de su propia interfaz, como un “Social Media Opinion Simulation System”. Su README dice que recibió apoyo estratégico e incubación de Shanda Group y que el motor de simulación es **OASIS** (camel-ai), un simulador de redes sociales con agentes de lenguaje, de código abierto bajo Apache-2.0.

Funciona así: subís un documento (informe, noticia, texto) y describís en lenguaje natural lo que querés simular. MiroFish extrae de ahí entidades, crea personajes (agentes) con personalidad y memoria, los pone a interactuar en plataformas simuladas y al final genera un reporte. Después podés conversar con los agentes.

SECCIÓN 02
CÓMO INSTALARLO

Node, Python, dos claves de API y cuatro comandos.

REQUISITO	DETALLE (SEGÚN EL README Y EL CÓDIGO)
Herramientas	Node.js 18 o superior, Python 3.11 o 3.12 (no 3.13) y el gestor **uv**.
Clave de LLM	Cualquier API con formato OpenAI. El README recomienda el modelo **qwen-plus** por la plataforma Bailian de Alibaba y advierte que el consumo es alto: probá primero con menos de 40 rondas.
Clave de Zep	**Zep Cloud** guarda el grafo de conocimiento. El README dice que la cuota gratis mensual alcanza para uso simple. El código rechaza una instancia propia de Zep (`ZEP_API_URL`): solo funciona con Zep Cloud.

DESDE EL CÓDIGO FUENTE (RECOMENDADO POR EL README)

```
cp .env.example .env
npm run setup:all
npm run dev
```

Antes de arrancar, editá `.env` con `LLM_API_KEY`, `LLM_BASE_URL`, `LLM_MODEL_NAME` y `ZEP_API_KEY`. La interfaz queda en `http://localhost:3000` y la API en el puerto 5001. Con Docker: `docker compose up -d` (usa la imagen `ghcr.io/666ghj/mirofish` y el mismo `.env`).

SECCIÓN 03
QUÉ HACE

Cinco pasos, de tu documento al reporte y a las entrevistas.

PASO	QUÉ PASA
1. Grafo	Tu documento (PDF, MD o TXT, hasta 50 MB) se parte en bloques y se envía a Zep Cloud para construir un grafo de entidades y relaciones. Una IA genera antes la estructura (ontología) que guía esa extracción.
2. Entorno	Cada entidad se convierte en un agente con perfil, edad, género, país y MBTI “aparentes”, y una biografía y personalidad generadas por la IA. También se generan horarios de actividad, frecuencia de publicación, un evento inicial y los temas calientes.
3. Simulación	Los agentes actúan en paralelo en dos plataformas simuladas, una estilo Twitter (publicar, dar me gusta, republicar, citar, seguir) y otra estilo Reddit (publicar, comentar, votar, buscar, silenciar, entre otras). La simulación avanza por rondas y la actividad se puede ver en vivo y detener.
4. Reporte	Un “ReportAgent” redacta un reporte de 2 a 5 secciones consultando el grafo con herramientas de búsqueda y de entrevista, y deja un registro de cada paso que dio.
5. Interacción	Podés chatear con el ReportAgent, con cualquier agente, o enviar una pregunta (encuesta) a varios agentes a la vez.

SECCIÓN 04
¿SIRVE PARA LANDINGS?

Lo que probar una landing necesita, frente a lo que MiroFish hace.

TEMA	LO QUE ENCONTRÉ EN EL REPOSITORIO
Entrada	Acepta archivos PDF, MD y TXT. No encontré soporte para URL, HTML ni capturas de pantalla; los agentes trabajan con texto.
Qué simula	Conductas de red social: publicar, comentar, votar, republicar, seguir. No simula clics, scroll, formularios ni conversión.
Quiénes son los agentes	Personajes generados por IA a partir de las entidades de tu documento, no visitantes reales ni datos de tu analítica.

TEMA	LO QUE ENCONTRÉ EN EL REPOSITORIO
Qué es una “predicción”	El prompt del reporte indica que lo que dicen y hacen los agentes “es la predicción” del comportamiento futuro de las personas. No encontré en el código pruebas de exactitud (backtests o benchmarks) que respalden esa suposición.
Horarios	La IA genera los horarios según el público objetivo; si falla, el valor por defecto es una rutina china (pico entre las 19 y las 22 h, madrugada casi sin actividad).
Idioma	Se le puede pedir a la IA que responda en español (`languages.json` lo lista), pero los textos de la interfaz solo existen en inglés y chino, y arranca en chino por defecto.

MI EVALUACIÓN PARA LANDINGS Y DESARROLLOS

MiroFish está hecho para simular discusión y opinión pública sobre un tema, no para medir si una página convence o convierte. Lo más cercano a probar una landing sería subir su texto como documento semilla y usar la función de encuesta a los agentes para ver cómo reaccionan al mensaje. Eso da reacciones simuladas por un modelo de lenguaje, útiles como hipótesis o para detectar objeciones, pero no son evidencia de lo que harían tus visitantes. Para saber si “a las personas les va a gustar”, las pruebas con personas reales (analítica, pruebas A/B, grabaciones de sesiones, entrevistas con usuarios) siguen siendo lo que decide. Con los desarrollos pasa lo mismo: sirve para explorar cómo se discutiría un lanzamiento o un cambio, no para comprobar que el software funciona.

SECCIÓN 05
LO BUENO

Flujo completo, abierto y con entrevistas a los agentes.

- **Todo en una sola herramienta:** grafo, personajes, simulación, reporte y chat, con interfaz web y API propias.
- **Podés interrogar a los agentes** uno por uno, en lote o a todos a la vez, y consultar el historial de entrevistas.
- **El reporte deja rastro:** guarda un registro paso a paso de lo que hizo el ReportAgent, y las herramientas de búsqueda sobre el grafo te permiten revisar de dónde sale cada afirmación.
- **Código abierto con demos oficiales:** una demo en línea y dos videos (una simulación de opinión pública y un final alternativo de “Sueño en el pabellón rojo”). La página de GitHub muestra 80.3k estrellas, 12.2k forks y 322 commits.
- **Pruebas automáticas** en el backend (18 archivos de pruebas) y despliegue con Docker.

SECCIÓN 06
RIESGOS Y LÍMITES

Lo que conviene saber antes de usarlo con algo importante.

RIESGO	QUÉ PASA	CÓMO REDUCIRLO
Costo	El README avisa que el consumo de la API de LLM es alto y recomienda empezar con menos de 40 rondas.	Corré primero una simulación corta y mirá tu consumo.
Datos a terceros	Los bloques de tu documento se envían a Zep Cloud, la actividad de la simulación también se escribe ahí, y los textos pasan por el proveedor de LLM que configures.	No subas información de clientes sin su permiso.
Servidor abierto	El backend escucha en `0.0.0.0` por defecto, permite CORS desde cualquier origen en `/api/*`, trae una `SECRET_KEY` por defecto y no encontré autenticación en su API. La imagen de Docker arranca en modo desarrollo.	Usalo solo en local o en una red privada, nunca expuesto a internet.
Licencia AGPL-3.0	Si modificás el programa y otras personas lo usan por red, la sección 13 exige ofrecerles el código fuente de tu versión.	Antes de ofrecerlo a clientes como servicio, revisá la licencia con un abogado.
Resultados	Son texto generado por un modelo; el repositorio no incluye validación de exactitud y el proyecto está en versión 0.1.	Tomalos como hipótesis y contrastalos con datos reales.
Dependencia de Zep	Solo funciona con Zep Cloud y con versiones fijas de camel-oasis (0.2.5) y camel-ai (0.2.78).	Instalalo en un entorno aislado y fijá las versiones.

CÓMO LO USO YO

Uso MiroFish para testear webs y ver las landings, a ver si a las personas les va a gustar lo que armo. De forma parecida lo uso también en mis desarrollos.

SIGUIENTE PASO

Instalalo en local, corré una simulación corta (menos de 40 rondas) con un texto sin datos de clientes y leé el reporte y las respuestas de los agentes. Después comparalo con la reacción de personas reales antes de tomar decisiones.

IMPORTANTE: LA INFORMACIÓN PUEDE VARIAR

Todo lo de esta guía (versión, estrellas y forks, requisitos, licencias, funciones y riesgos) se revisó el **23 de octubre de 2026**. MiroFish está en versión 0.1 y se actualiza con frecuencia, por lo que a la fecha en que la leas puede haber cambiado. Los requisitos y servicios (por ejemplo Zep Cloud y el proveedor de LLM) también pueden variar según la región. Confirmá siempre en el repositorio oficial antes de instalar o decidir. Esta guía es informativa y no es asesoría legal.

Basada en la lectura del repositorio oficial 666ghj/MiroFish (README, código del backend y del frontend, configuración, licencia y textos de la interfaz), su página de GitHub y la página del proyecto OASIS (camel-ai). Preparada por Neiro Agency.